Q1. In a RAG pipeline, "late interaction" models such as ColBERT differ from bi-encoder models in that:

Answer: B

Q2. Document summarization as a pre-processing step in RAG ("summary indexing") is useful when:

Answer: B

Q3. In an LLM-based multi-agent system, the "orchestrator" agent's primary responsibility is to:

Answer: B

Q4. XLNet differs from BERT in its pre-training approach by:

Answer: B

Q5. The primary limitation of using a decoder-only model for extractive QA compared to an encoder-only model is:

Answer: B

Q6. In the original transformer (Vaswani et al., 2017), positional encoding uses sinusoids of different frequencies because:

Answer: B

Q7. Multi-Query Attention (MQA) reduces memory and inference latency compared to MHA by:

Answer: B

Q8. Curriculum learning in the context of LLM pre-training refers to:

Answer: A

Q9. In LLM inference, "key-value cache eviction" strategies (e.g., H2O, SnapKV) are used to:

Answer: B

Q10. The Adam optimiser's bias-correction step (dividing by 1 - beta^t) is necessary because:

Answer: B

Q11. Continued pre-training (domain-adaptive pre-training) differs from instruction fine-tuning in that:

Answer: B

Q12. ORPO (Odds Ratio Preference Optimization) combines supervised fine-tuning and preference alignment by:

Answer: B

Q13. Weight clustering (k-means quantization) differs from uniform quantization in that:

Answer: B

Q14. The main benefit of FP8 training (as used in H100 GPU Tensor Cores) over INT8 training is:

Answer: B

Q15. A translation model is evaluated with BLEU on a test set and scores 42. Without additional context, this number:

Answer: B

Q16. The WMT shared task evaluation campaigns are significant in MT research because:

Answer: C

Q17. Reference-free evaluation metrics for machine translation (e.g., COMET-QE, CometKiwi) are advantageous in production settings because:

Answer: B

Q18. Layer normalization is applied differently in Pre-LN (GPT-2 style) vs Post-LN (original transformer). Pre-LN is preferred in large models because:

Answer: B

Q19. In the context of quantized LLM deployment, "dequantization overhead" at inference time refers to:

Answer: B

Q20. In RAG, the "lost in the middle" phenomenon refers to:

Answer: B

Coding Questions:

Problem Statement

Given an integer array nums, for each element nums[i], count the number of elements to its right that are strictly smaller than nums[i].

Return an integer array counts where counts[i] is the count for nums[i].

You must solve this using binary search on a dynamically maintained sorted list - do not use a brute-force O(n^2) nested loop as your primary solution.

Constraints

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4
- Expected time complexity: O(n log n)

Open Test Cases (Visible to Candidate)

```
Input:  nums = [5, 2, 6, 1]

Output: [2, 1, 1, 0]
```

```
Input:  nums = [1]

Output: [0]
```

Solution:

```
from bisect import bisect_left, insort_left

def countSmallRight(nums):

    counts = []

    dynamic_sorted_list = []


    for num in reversed(nums):

        idx = bisect_left(dynamic_sorted_list, num)

        counts.append(idx)

        insort_left(dynamic_sorted_list, num)

    return counts[::-1]


nums = [5,2,6,1]

nums1 = [1]

print(countSmallRight(nums))

print(countSmallRight(nums1))
```

Approach:

1. The problem is stating for each element in the array, the count of numbers to it's right that are smaller than it.
2. Initially we need to process the array from right to left, maintaining a sorted list of elements seen so far
3. For each element we use binary search to find it's intersection index in the dynamically sorted list. This index is the count of smaller elements to it's right.

We used predefined functions from library bisect land those functions are bisect_left(list, num), which finds the index where num must be inserted to keep list in a sorted manner. Additionally we also used function insort_left(list, num): which is going to insert num into list which is in sorted manner at the correct position to keep it sorted. Both of these functions

are going to operate in O(logn) time complexity. The time complexity of the solution is O(nlogn) since we used for loop to iterate through n elements and the predefined functions are going to take logn time complexity. Apart from this space complexity is O(n).

Coding Q2 - Longest Repeating Character Replacement

Problem Statement

You are given a string s consisting of uppercase English letters and an integer k.

You may choose at most k characters in the string and replace them with any uppercase letter.

After performing at most k replacements, find the length of the longest substring that contains only one distinct character.

Constraints

- 1 <= s.length <= 10^5
- s consists of uppercase English letters only
- 0 <= k <= s.length
- Expected time complexity: O(n)

Open Test Cases (Visible to Candidate)

```
Input:  s = "ABAB",  k = 2

Output: 4



Input:  s = "AABABBA",  k = 1

Output: 4
```

Solution:

```
from collections import defaultdict


def charReplacement(s,k):
```

```
    count = defaultdict(int)
    left = 0
    max_count = 0
    result = 0

    for right in range(len(s)):
        count[s[right]] += 1
        max_count = max(max_count, count[s[right]])

        while (right - left + 1) - max_count > k:
            count[s[left]] -= 1
            left += 1

        result = max(result, right-left + 1)

    return result



s = "ABAB"
k = 2
s1 = "AABABBA"
k1 = 1
print(charReplacement(s,k))
print(charReplacement(s1,k1))
```

Approach:

1. The problem is stating that we need to findout the longest substring where we can replace at most k chars to make all characters are same.
2. Our approavch is to use a sliding window with a frequency counter

3. For each window, we need to keep track the count of the most frequenct char. The number of replacements needed is (window-size – max-freq)
4. If replacements needed > k, we need to shrink the window from left
5. This ensures the time complexity O(n) and since we did not use any addiional space our space complexity is O(1)

Coding Q3 - Subarray Sum Equals K

Problem Statement

Given an integer array nums and an integer k, return the total number of contiguous subarrays whose elements sum to exactly k.

The array may contain negative numbers.

You must solve this in O(n) time - do not use a brute-force O(n^2) approach as your primary solution.

Constraints

- 1 <= nums.length <= 2 x 10^4
- -1000 <= nums[i] <= 1000
- -10^7 <= k <= 10^7
- Expected time complexity: O(n)

Open Test Cases (Visible to Candidate)

```
Input:  nums = [1, 1, 1],  k = 2

Output: 2



Input:  nums = [1, 2, 3],  k = 3

Output: 2
```

Solution:

```
from collections import defaultdict

def subarray(nums, k):
    prefix_count = defaultdict(int)
```

```
    prefix_count[0] = 1

    current_sum = 0
    result = 0

    for num in nums:
        current_sum += num
        result += prefix_count[current_sum - k]
        prefix_count[current_sum] += 1

    return result

nums = [1,1,1]
k = 2
nums1 = [1,2,3]
k1 = 3
print(subarray(nums,k))
print(subarray(nums1, k1))
```

Approach:

1. The problem is stating that count the number of subarrays whose sum is k.
2. Our approach includes we are going to use prefix sums and a hashmap to store the frequency of prefix sums seen so far.
3. For each element we can compute the running sum (prefix sum). If the current sum – k exists in the hashmap, it means there is a subarray ending at the current index which sum is k.
4. We increment the count by the frequency of currentsum -k in the hashmap
5. Finally, we are going to update the hashmap with the current prefix sum.
6. Time Compexity is O(n) since we iterate through the array once and all hashmap operations are O(1)
7. Space Complexirty is O(n) since we stored the prefix sums in the hashmap.